

Nombres:

- Carla Daniela Martínez
- Cesar Alejandro Contreras
- Emanuel Velandia
- Alejandro Morales

LABORATORIO – Gestión de Estudiantes con Spring Boot**Construcción de un servicio REST para la gestión de estudiantes usando Spring Boot**

En este laboratorio, debe implementar un **servicio web REST** utilizando **Spring Boot**, con el fin de gestionar un conjunto de estudiantes en una aplicación backend. El propósito es reforzar conceptos de controladores REST, modelos de datos, servicios de negocio y manejo de colecciones en memoria.

El servicio permitirá realizar las operaciones básicas de un CRUD (**Crear, Leer, Actualizar y Eliminar**) sin utilizar todavía bases de datos externas.

Los datos serán almacenados en una colección interna para facilitar la comprensión de la arquitectura inicial.

Contexto del ejercicio:

La Universidad desea implementar un módulo básico para registrar estudiantes en un prototipo de plataforma académica. Por ahora, solo se necesita un servicio backend que permita registrar estudiantes y consultar su información a través de rutas HTTP. Este laboratorio sirve como base para futuros módulos como matrículas, asignaturas o notas.

El objetivo principal es que el estudiante comprenda el flujo entre:

- **Controlador REST**
- **Servicio de lógica de negocio**
- **Modelo de datos**
- **Uso de rutas HTTP (GET, POST, PUT, DELETE)**

Para controlar el modelo se deben usar:

- Crear controladores REST usando `@RestController` y `@RequestMapping`.

`@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@Service`, Manejo de listas en memoria (`ArrayList`).

- Implementar modelos de datos en Java.
- Separar responsabilidades mediante un servicio de negocio (`@Service`).

- Crear endpoints que reciban y respondan datos en formato JSON.
- Usar Postman o navegador para consumir servicios REST.
- Comprender el funcionamiento básico de un CRUD antes de integrar bases de datos.

Requerimientos funcionales:

El sistema debe permitir:

1. **Listar todos los estudiantes (GET).**
2. **Buscar un estudiante por ID (GET).**
3. **Registrar un nuevo estudiante (POST).**
4. **Actualizar los datos de un estudiante existente (PUT).**
5. **Eliminar un estudiante por ID (DELETE).**

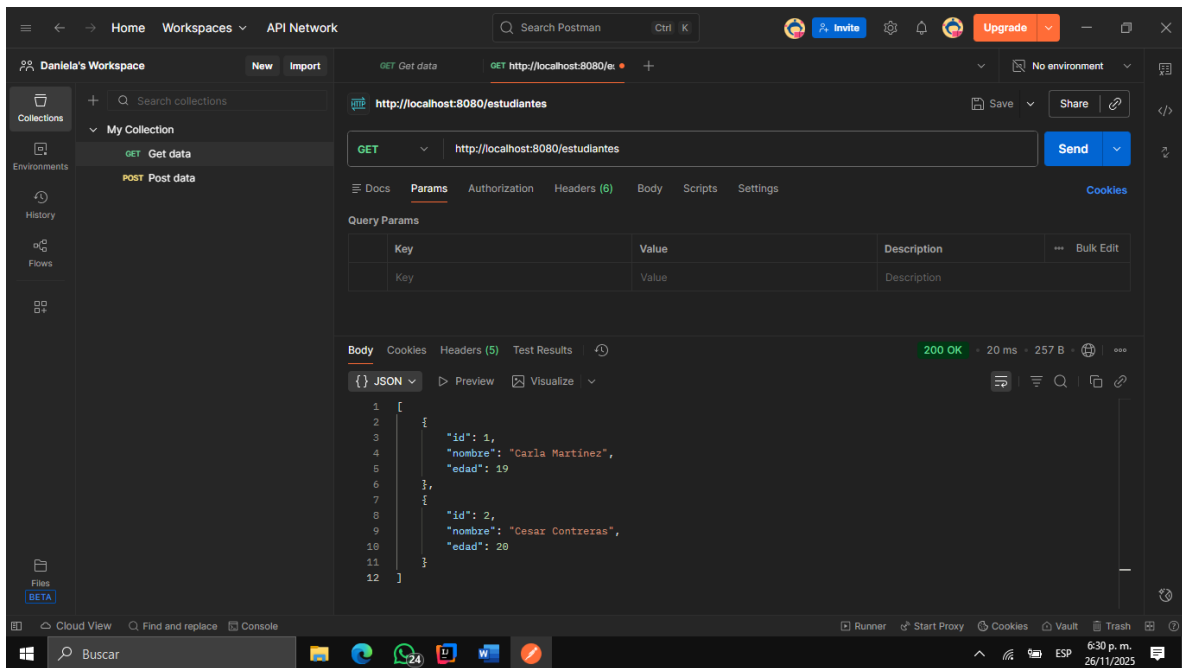
Cada estudiante debe contener como mínimo:

- id (entero)
- nombre (cadena de texto)
- edad (entero)

Los datos iniciales deben incluir **al menos dos estudiantes de ejemplo** cargados en memoria.

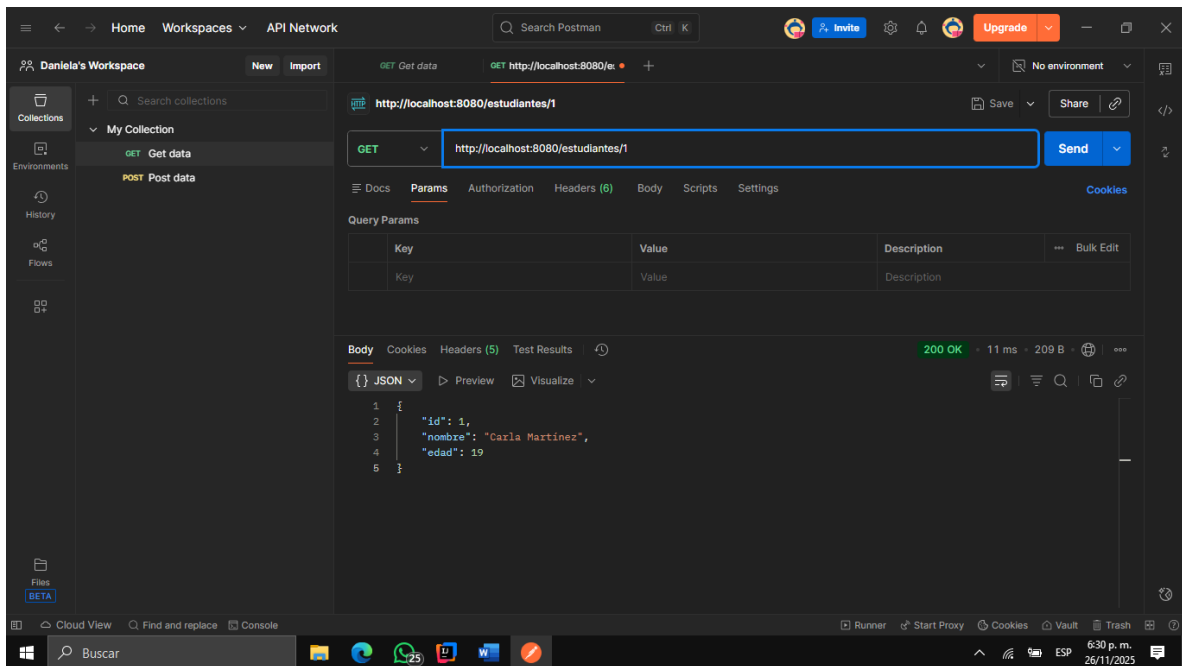
OJO:

- Código fuente completo del proyecto con las siguientes capas:
model
service
Controller
1. Capturas de pantalla de las pruebas de los endpoints:
 - Listar estudiantes

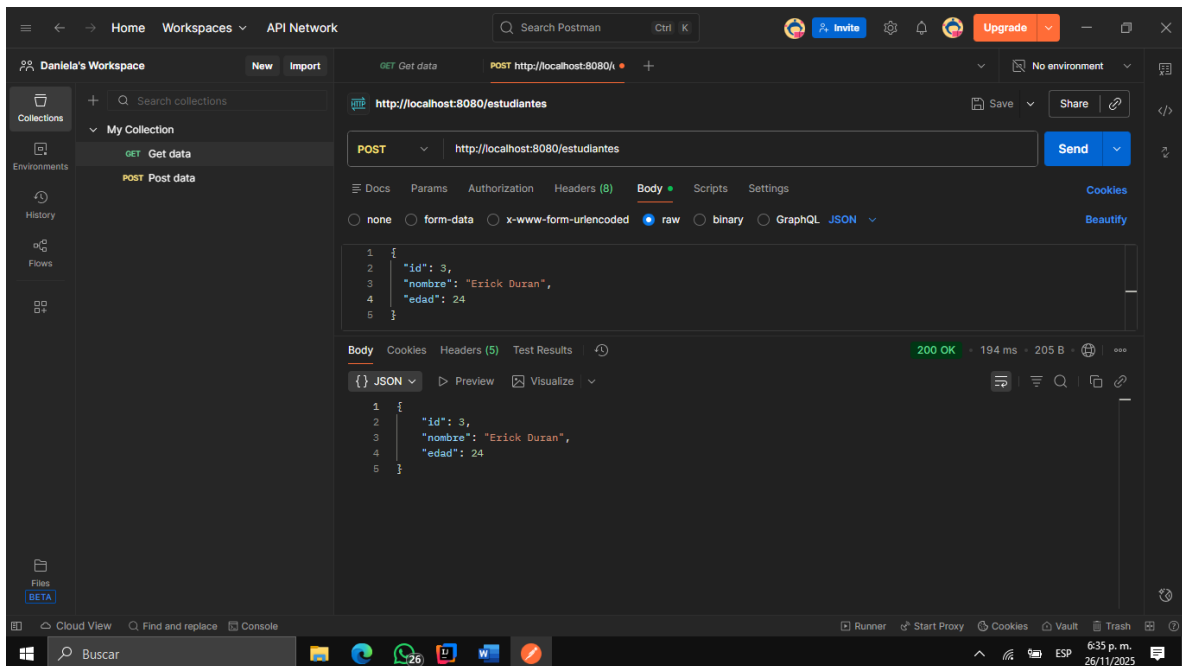


○ Buscar por ID

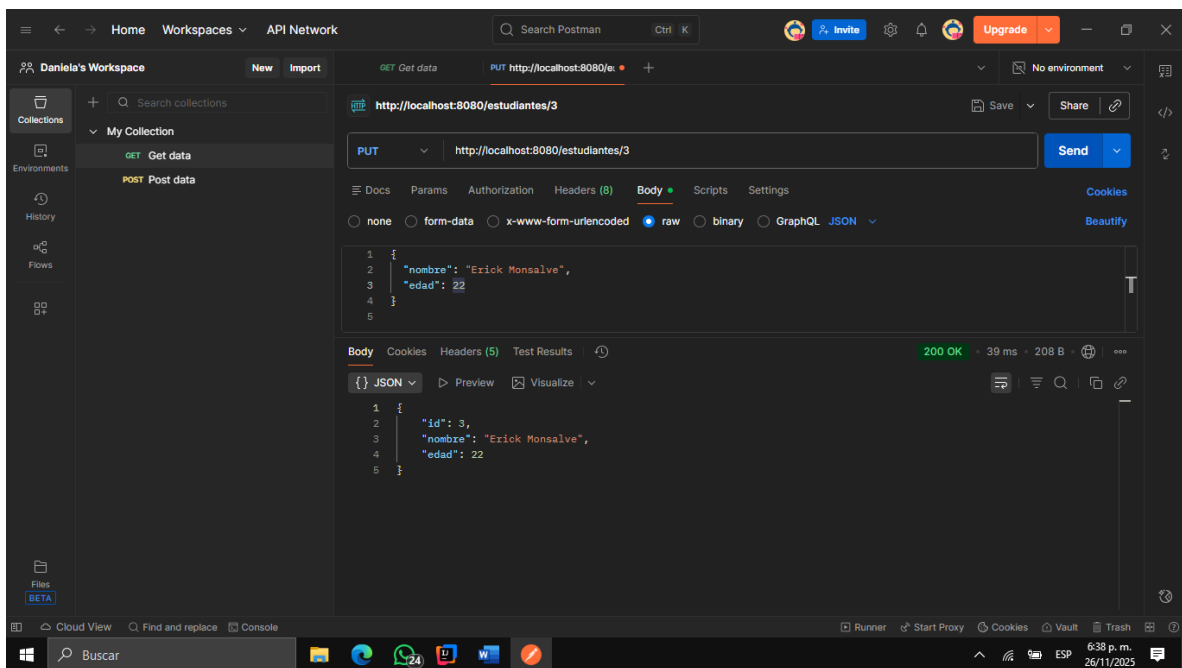
ID = 1



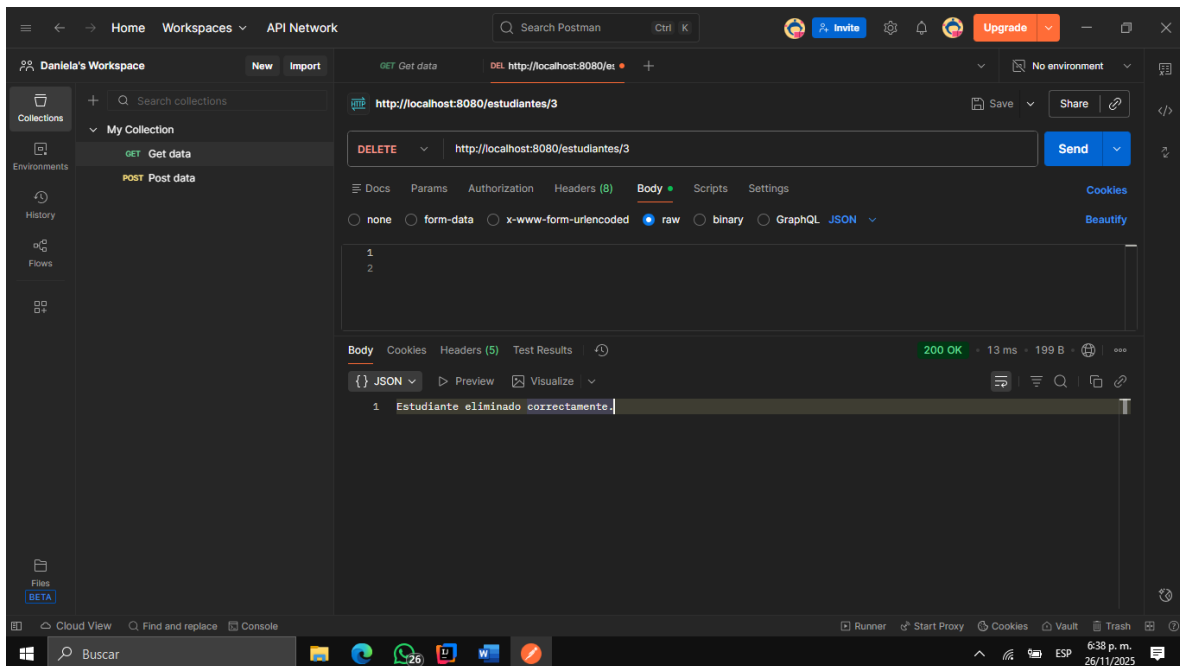
○ Crear estudiante



○ Actualizar estudiante



○ Eliminar estudiante



2. Documento corto respondiendo:

- ¿Qué ventajas y desventajas tiene usar listas en memoria?

R/

Ventajas

Trabajar con listas en memoria (como ArrayList) es ideal para un laboratorio porque permite montar el CRUD rápido y entender bien la arquitectura por capas: Controller → Service → Model. No depende de una base de datos ni de configuraciones extra, entonces nos podemos concentrar en las rutas GET, POST, PUT y DELETE, en cómo se reciben datos en JSON y cómo se devuelven respuestas. Para poca cantidad de datos funciona muy bien y es muy rápido porque todo está en RAM.

Desventajas

La desventaja más clara es que no hay persistencia: si se reinicia el Spring Boot, se borra todo lo que se registro. También hay temas de consistencia: la persona tiene que manejar reglas como IDs únicos, validaciones y evitar duplicados. Además, para buscar por ID normalmente se recorre la lista completa (búsqueda lineal), entonces con muchos registros se vuelve poco eficiente. Y si el proyecto crece, la lista se queda corta porque no soporta consultas avanzadas (filtros, paginación, ordenamiento real) de forma robusta.

- ¿Qué ocurriría si hubiese múltiples usuarios accediendo al sistema al mismo tiempo?

R/ Como Spring Boot atiende varias peticiones al tiempo (multi-hilo), una lista en memoria puede generar problemas de concurrencia. Por ejemplo:

- Dos usuarios hacen POST al mismo tiempo y si el ID se genera “a mano”, podría haber choque o comportamientos raros.
 - Uno hace PUT mientras otro hace DELETE al mismo estudiante y puede quedar inconsistente (uno actualiza algo que ya fue borrado).
 - Puede pasar que un usuario consulte (GET) justo cuando otro está modificando la lista, y la respuesta salga incompleta o distinta a lo esperado.
- ¿Qué mejorarías si usaras una base de datos real?

R/ Con una base de datos real se gana lo principal: persistencia (los estudiantes no se pierden al reiniciar). También mejora la integridad de datos porque la BD puede asegurar IDs únicos y reglas más fuertes. Las consultas también serían más eficientes: buscar por ID es directo, se pueden hacer filtros (por edad, por nombre), paginación, ordenamiento, etc.